

2026-03-23 Spider Reference Stepper Readthrough

Executive Summary

这次 spider gait 的实现不再以“调一组看起来更像的参数”为主，而是明确对齐一个完整的开源实现：PhilS94/Unity-Procedural-IK-Wall-Walking-Spider。[1] 我这次实际照着读完并落地的不是它全部的 wall-walking / CCD IK / Unity 控制器，而是对当前 godot_citys 最关键的四段 stepping 管线：step desire -> tetrapod group gating -> anchor/overshoot/prediction -> arc stepping，再用蜘蛛运动学论文校正两段腿代理比例与步态直觉。[1][2]

Reference Project

- 项目名称：PhilS94/Unity-Procedural-IK-Wall-Walking-Spider
- 项目地址：<https://github.com/PhilS94/Unity-Procedural-IK-Wall-Walking-Spider>
- 项目简介：一个 Unity 开源程序化蜘蛛项目，核心展示的是 per-leg step check、tetrapod gait scheduler、anchor/overshoot/prediction 脚点规划，以及可见的 swing stepping；仓库还包含 wall-walking / ceiling-walking / CCD IK，但当前 godot_citys 这轮主要吸收的是 stepping 调度与脚点预测这条主链。[1]

Key Findings

- 参考项目的核心不是“相位表”，而是 **step manager**：每条腿先独立判断“想不想迈步”，再由统一 manager 决定“现在准不准迈”。[1]
- 新脚点不是直接用 **anchor**，而是 **anchor + overshoot + body-travel prediction**：这比“当前脚点跟着全局相位抖动”更像活物。[1]
- 脚在迈步期间不应立即修改支撑脚点：参考实现里旧 target 与新 target 之间有一段拱线过渡，真正 plant 之后才切到新脚点。[1]
- 第二轮最该对齐的是 **scheduler**，而不是继续盲调腿比
例：IKStepManager.AlternatingTetrapodGait() 不是纯相位窗口，而是 currentGaitGroup + nextSwitchTime + averageStepTime 的显式 timer scheduler。[1]
- 论文层面的比例约束依然重要：就算 stepping 管线正确，腿段如果还是中点折叠、近端远端近等长，视觉依然会假。[2]

Detailed Analysis

1. 参考项目到底在做什么

PhilS94 的仓库里，步态逻辑主要在两个类：

- `IKStepManager.cs`
- `IKStepper.cs`

我这次是按下面的顺序读的。

第一层是 `IKStepper.stepCheck()`。[1] 它不是拿一个全局时钟直接判定“这条腿该 lift 了”，而是先看三件事：

1. 当前是不是已经在 stepping
2. 已经静止太久了没有
3. 当前脚点是不是已经偏离默认 anchor 太多，或者 IK 误差太大

这一步非常关键，因为它决定的是“腿有没有迈步欲望”，不是“腿处在哪个相位”。

第二层是 `IKStepManager`。[1] 它不是让每条腿各迈各的，而是统一做 gating。这个项目给了两类策略：

- queue 模式
- alternating tetrapod gait 模式

我这次对 `godot_citys` 选的是后者，因为它更贴近仓库当前 spider lab 的目标，也和现有 8 腿分组结构更容易接上。

2. 参考项目是怎么算新脚点的

`IKStepper.Step()` 和 `calculateDesiredPosition()` 是这次最值得抄的部分。[1]

它的逻辑不是“把脚点沿运动方向随便往前扔一点”，而是：

1. 找到当前脚点
2. 找到这条腿在 body local frame 下的默认 anchor
3. 把当前脚点投影到 anchor 高度平面
4. 从这个投影点朝 anchor 画线
5. 用一个 `overshoot multiplier` 把这条线再延长一点
6. 再加上 spider 在 step time 内预计会走出去的距离
7. 最后再用射线/球射线找到真实地表脚点

对当前仓库来说，真正值钱的是第 3 到第 6 步。因为我们之前的问题恰好就在这里：脚点基本还是被相位直接驱着走，而不是被“默认站姿 anchor + 运动预测”驱着走。

3. 参考项目的 stepping 为什么更像活物

参考实现中，step 期间脚不会瞬移到新 target，而是通过一段曲线从旧 target 过渡到新 target。[1] 这件事的重要性比看上去大得多，因为它同时保证了两件事：

- 支撑脚点在 plant 前不会立刻变掉
- 视觉上的脚尖轨迹是一段真正的拱线，而不是“锁点突然换位置”

我这次把这一点直接翻译成了 `godot_citys` 里的：

- `step_goal_world_position`
- `step_prediction_world_position`
- `default_anchor_world_position`
- `display_foot_world_position`
- `step_progress`

这几个字段现在都能从 spider debug state 里直接看到。

4. 这次在 `godot_citys` 里具体怎么对齐

这次没有把整个 `PhilS94` 项目生搬硬套进来，因为当前仓库并不需要它的 wall-walking / collider fake gravity / Unity CCD 链。[1] 但我对齐了它最核心的 stepping 主链：

1. Step Desire

- 每条腿根据当前 locked foothold 和默认 anchor 的偏差判断是否“想迈步”

2. Tetrapod Group Gating

- 仍然使用两组 4 腿的 alternating window，而不是 8 条腿各自按相位切

3. Anchor/Overshoot/Prediction

- 新脚点由默认 anchor、overshoot multiplier、角色速度预测共同决定

4. Arc Stepping

- 脚尖显示位置在 start 和 goal 之间走拱线，直到 plant

这次真正改动落在：

- `city_game/world/creatures/arthropods/CitySpiderCrawler.gd`
- `tests/world/test_spider_crawler_reference_step_contract.gd`

其中 `CitySpiderCrawler.gd` 现在明确暴露 `step_controller_id = reference_anchor_prediction_v1`，表示这只 spider 已经不再是“纯手调相位蜘蛛”。

5. 第二轮为什么要继续对齐 scheduler

第一轮对齐完之后，蜘蛛已经不再是“纯相位摆腿”，但离参考实现还有一个关键差距：当前仓库当时仍然用 `_reference_step_clock` 去推 A/B 组窗口，本质上还是 phase-window gating；而 `PhilS94` 的 `IKStepManager.AlternatingTetrapodGait()` 真正做的是另一件事。[1]

它先维护两个显式状态：

- `currentGaitGroup`
- `nextSwitchTime`

然后只在 `Time.time >= nextSwitchTime` 的瞬间切组、统一计算新组的 `averageStepTime`，并把这个时间同时赋给该组所有本轮要迈步的腿。[1]

这件事的价值不只是“更像参考实现”，而是直接决定 gait 的时间结构：group cadence 由**当前速度下的动态 step time**决定，而不是由一个静态 phase-duration 决定。对视觉来说，这会带来两个可感知变化：

- 当速度升高时，A/B 组切换会真实变快，而不是继续按固定相位半周期切换
- 同一组里真正起步的腿会共享同一轮 step duration，不会出现“看起来是一组，但每条腿像在各走各的”

这也是为什么第二轮实现里，我把 spider debug state 继续扩展为：

- `step_scheduler_id = reference_tetrapod_timer_v2`
- `active_step_group_id`
- `next_group_switch_time`
- `group_step_time_seconds`
- `group_switch_count`
- `step_clock_seconds`

并新增 `test_spider_crawler_reference_scheduler_contract.gd`，直接卡住“两次组切换之间的时间间隔必须跟上一轮 group step time 对齐”。这一步的目标不是再做一个“更平滑的效果”，而是把 reference repo 真正值钱的 scheduler contract 也落成仓库资产。[1]

Areas of Consensus

- 开源参考实现和论文都支持：蜘蛛 gait 不能只靠统一 phase-clock 来解释。[1][2]
- 默认 anchor 与预测步点是程序化多足步态的关键抽象。[1]
- 真正的 swing arc 对“像不像活物”非常重要。[1]

Areas of Debate

- **是否必须把完整 CCD IK 也一起搬过来**：对当前 lab 目标不是必须；当前先把 stepping 管线对齐更划算。[1]
- **tetrapod group 是否要继续完全同步**：参考实现允许整组同窗 stepping，但自然感更强的版本以后可以再加 per-leg timing jitter。[1]
- **timer-based scheduler 是否已经足够“像活物”**：还不够。它只是把 cadence 主链对齐了；后续若继续提升真实感，更该下钻 body sway、3-link visual proxy 与更真实的 foothold workspace。[1][2]

Sources

[1] PhilS94. “Unity-Procedural-IK-Wall-Walking-Spider.” GitHub repository. 这是这次直接逐文件阅读并对齐的主参考，实现价值高，但不是 peer-reviewed source。 <https://github.com/PhilS94/Unity-Procedural-IK-Wall-Walking-Spider>

[2] Wilson RS. et al. “Biomechanics of octopedal locomotion: kinematic and kinetic analysis of the spider *Grammostola mollicoma*.” *Journal of Experimental Biology* 214(20), 2011. Peer-reviewed biomechanics paper，用来约束腿段比例与 gait realism 判断。
<https://journals.biologists.com/jeb/article/214/20/3433/10466/Biomechanics-of-octopedal-locomotion-kinematic-and>

Gaps and Further Research

- 这次仍未把 PhilS94 的 wall-walking / ceiling logic 接进当前 lab
- 还没有把完整 IK solve error 当作 step desire 输入；当前先用 anchor drift 近似
- 如果下一轮继续做，最值钱的是：
 - 把 per-leg workspace exhaustion 做成真正 contract
 - 给 tetrapod group 加小幅 timing jitter
 - 把 2-link visual proxy 升成 3-link spider proxy